

Aula 06 - 24/03/2026

Herança, polimorfismo, classes abstratas e interfaces

Sumário

- Herança: Reutilização e Especialização
 - Conceito de Superclasse e Subclasse (extends)
 - A chamada super() em construtores
- Polimorfismo: Múltiplas Formas, Um Único Método
 - Sobrescrita de Métodos (@Override)
- Classes Abstratas: O Molde Incompleto
 - Quando e por que usar? Métodos abstratos
- Interfaces: O Contrato de Comportamento
 - Diferenças cruciais e implementação múltipla
- Design Patterns em Ação
 - Strategy (com Interfaces)
 - Template Method (com Classes Abstratas)
- Exercícios Práticos

Herança: Reutilização e Especialização

- **O que é?**

- Herança é o pilar da reutilização. Permite que uma classe (filha ou subclasse) herde atributos e métodos de outra classe (mãe ou superclasse).

- **Sintaxe:**

- Usamos a palavra-chave `extends`.

- **Java**

```
public class Gerente extends Funcionario {  
    // Gerente herda tudo de Funcionario  
}
```

- **Para que serve?**

- **Generalização/Especialização:** Criar uma hierarquia onde classes mais específicas (como Gerente) especializam uma classe mais genérica (como Funcionario).
- **Evitar Duplicação de Código:** Atributos como nome e cpf não precisam ser reescritos.

A Chamada `super()`

- **Construtores não são herdados!**

- Quando um objeto da subclasse é criado, o construtor da superclasse precisa ser chamado primeiro para inicializar os atributos herdados.

- **Como fazer?**

- Use `super()` para invocar o construtor da classe mãe. Deve ser a primeira linha do construtor da subclasse.

Exemplo

J Funcionario.java

src > oo > J Funcionario.java > Funcionario

```
1  package oo;
2
3  public class Funcionario {
4      protected String nome;
5      protected double salario;
6
7      public Funcionario(String nome, double salario) {
8          this.nome = nome;
9          this.salario = salario;
10     }
11 }
```

J Gerente.java 1

src > oo > J Gerente.java > Gerente

```
1  package oo;
2
3  public class Gerente extends Funcionario {
4      private String area;
5
6      public Gerente(String nome, double salario, String area) {
7          super(nome, salario); // Chama o construtor de Funcionario
8          this.area = area;
9      }
10 }
```

Polimorfismo: Múltiplas Formas

- **O que é?**

- Do grego, "muitas formas". É a capacidade de um objeto ser tratado como uma instância de sua própria classe, de sua superclasse ou de uma interface que implementa.

- **A Mágica da Sobrescrita (@Override)**

- O polimorfismo brilha quando sobrescrevemos métodos. A JVM decide, em tempo de execução, qual implementação do método chamar, com base no tipo real do objeto.

Exemplo

J Funcionario.java

src > oo > J Funcionario.java > Funcionario > getBonificacao()

```
1 package oo;
2
3 public class Funcionario {
4     protected String nome;
5     protected double salario;
6
7     public Funcionario(String nome, double salario) {
8         this.nome = nome;
9         this.salario = salario;
10    }
11
12    public double getBonificacao() {
13        return this.salario * 0.10;
14    }
15 }
```

J Gerente.java 1 X

src > oo > J Gerente.java > Gerente

```
1 package oo;
2
3 public class Gerente extends Funcionario {
4     private String area;
5
6     public Gerente(String nome, double salario, String area) {
7         super(nome, salario); // Chama o construtor de Funcionario
8         this.area = area;
9     }
10    @Override // Boa prática para garantir a sobrescrita
11    public double getBonificacao() {
12        return this.salario * 0.15 + 1000;
13        /* No main:
14         Funcionario f = new Gerente("Ricardo", 10000.0, "TI");
15         System.out.println(f.getBonificacao());
16        */
17    }
18 }
```

Classes Abstratas: O Molde Incompleto

- **O que são?**

- Classes que não podem ser instanciadas. Servem apenas como um modelo (molde) para outras classes.

- **Quando usar?**

- Quando você tem um conceito genérico que não faz sentido existir sozinho. Ex: "Veículo" é um conceito, mas você dirige um "Carro" ou uma "Moto".
- Para garantir que subclasses implementem certos métodos.

- **Métodos Abstratos:**

- São métodos declarados sem corpo ({}). Obrigam as subclasses concretas a fornecerem uma implementação.

Exemplo

J Animal.java X

src > oo > J Animal.java >  Animal

```
1  package oo;
2
3  public abstract class Animal {
4      // Método concreto, herdado por todos
5      public void respirar() {
6          System.out.println(x:"Respirando...");
7      }
8
9      // Método abstrato, deve ser implementado pelas subclasses
10      public abstract void fazerBarulho();
11 }
```

Interfaces: O Contrato de Comportamento

- **O que são?**

- Um "contrato" 100% abstrato que define um conjunto de comportamentos (métodos) que uma classe deve ter. Uma classe implements uma interface.

- **Regras importantes:**

- Uma classe só pode extends uma outra classe.
- Uma classe pode implements várias interfaces (herança múltipla de tipos).
- São ideais para conectar partes do sistema que não têm uma relação de herança direta.

Exemplo

J Autenticavel.java ●

src > oo > J Autenticavel.java > Autenticavel

```
1 package oo;
2
3 public interface Autenticavel {
4     boolean autenticar(String senha);
5 }
```

J Gerente.java 1 ●

src > oo > J Gerente.java > Gerente > autenticar(String)

```
1 package oo;
2
3 public class Gerente extends Funcionario implements Autenticavel {
4     private String area;
5
6 > public Gerente(String nome, double salario, String area) { ...
10 @Override // Boa prática para garantir a sobrescrita
11 > public double getBonificacao() { ...
18
19     public boolean autenticar(String senha) {
20         return "1234".equals(senha);
21     }
22 }
```